CEG 3156: Computer Systems Design
(Winter 2026)

Prof. Rami Abielmona

Lab #2:
Single-Cycle MIPS Processor

Submitted by:
Mann Patel — 300363879
Surya Vasudev — 300358602

Date: March 18th, 2026

Table of Contents

1. Introduction 3
2. Design Overview 3
3. Component Design 3
   3.1. Program Counter Register (`pc_reg`) 3
   3.2. N-Bit Adder/Subtractor (`nBitAddSubUnit`) 3
   3.3. Instruction Memory (`instruction_memory`) 4
   3.4. Control Unit (`Control_Unit`) 4
   3.5. 2-to-1 Multiplexer (`nBitMux2to1`) 4
   3.6. Register File (`register_file`) 4
   3.7. Sign Extension Unit (`sign_extended`) 5
   3.8. ALU Control (`ALU_control`) 6
   3.9. N-Bit ALU (`nbit_ALU`) 6
   3.10. Data Memory (`data_memory`) 6
4. Datapath Interconnection 6
   4.1. Fetch Stage 6
   4.2. Decode Stage 6
   4.3. Execute Stage 7
   4.4. Memory and Write-Back Stage 7
   4.5. Output Debug Multiplexer 7
   4.6. Cycle-by-Cycle Trace 8
5. Timing Analysis and CPU Execution Time 8
   5.1. Critical Path Analysis 8
      5.1.1. Gate-Level Delay of the 8-Bit Ripple-Carry Adder 8
      5.1.2. ALU Worst-Case Delay 8
      5.1.3. Total Critical Path 8
   5.2. CPU Execution Time Calculation 9
6. Simulation Results 9
7. Design Challenges and Solutions 11
8. Conclusion 11

1. Introduction

The objective of this laboratory is to design, implement, and verify a single-cycle MIPS processor in VHDL. The processor supports a subset of the MIPS instruction set, including:

- **Memory-reference instructions:** `lw` (load word) and `sw` (store word)
- **Arithmetic/logic instructions:** `add`, `sub`, `and`, `or`, and `slt`
- **Control-flow instructions:** `beq` (branch if equal) and `jump` (unconditional jump)

The design uses **8-bit data paths** with **32-bit instruction widths**. A register file of eight 8-bit registers is addressed with 3 bits. The instruction memory is a 256×32 ROM, while the data memory is a 256×8 RAM.

2. Design Overview

The processor follows the MIPS single-cycle datapath executed within a single clock cycle:

1. **Fetch** : The program counter (PC) addresses the instruction memory; PC is incremented by 4.
2. **Decode** : The fetched 32-bit instruction is decoded; source registers are read and the control unit generates all control signals.
3. **Execute** : The ALU performs the operation dictated by the ALU control unit.
4. **Memory/Write-Back** : Data memory is accessed for load/store instructions; the result is written back to the register file.

3. Component Design

3.1. Program Counter Register (`pc_reg`)

The PC register is an 8-bit positive-edge-triggered register with synchronous reset.

Port	Width	Description
`i_clock`	1	Rising-edge clock
`i_reset`	1	Synchronous reset
`i_d`	8	Next PC value
`o_q`	8	Current PC value

Table 1: PC register — entity interface.

The register is constructed from eight D flip-flops, each gated by the common clock and reset signals.

3.2. N-Bit Adder/Subtractor (`nBitAddSubUnit`)

A ripple-carry adder/subtractor is used in two places:

1. **PC + 4 adder** — computes `pc_plus4 = pc_current + 0x04` (addition mode, `i_OpFlag = '0'`).
2. **Branch target adder** — computes `branch_target = pc_plus4 + branch_offset` (addition mode).

When `i_OpFlag = '1'` the unit performs subtraction via two's-complement inversion of the `i_Bi` input. Internally, the unit is built from n cascaded 1-bit full adders with XOR-based conditional inversion on the B input and the carry-in tied to `i_OpFlag`.

Instr.	RegDst	ALUSrc	Mem-toReg	Reg-Write	Mem-Read	MemWrite	Branch	Jump	ALUOp
R-type	1	0	0	1	0	0	0	0	10
`lw`	0	1	1	1	1	0	0	0	00
`sw`	×	1	×	0	0	1	0	0	00
`beq`	×	0	×	0	0	0	1	0	01
`j`	×	×	×	0	0	0	0	1	××

Table 2: Control signal truth table.

3.3. Instruction Memory (`instruction_memory`)

The instruction memory is a 256×32 synchronous ROM instantiated using the Altera LPM_ROM megafunction. It is initialized with a Memory Initialization File (`.mif`) containing the benchmark program. On each rising clock edge, the ROM registers the address and presents the corresponding 32-bit instruction on `q`.

Design note: The address input is connected to `pc_next` (rather than `pc_current`) so that the ROM's internal address register and the PC register sample the same next-address simultaneously on the rising edge. This ensures that `InstructionOut` reflects the instruction corresponding to the current PC value without additional combinational delay.

3.4. Control Unit (`Control_Unit`)

The main control unit is a combinational decoder that accepts the 6-bit opcode field `instruction[31:26]` and produces nine control signals:

The control unit is implemented with AND/OR gate networks derived from the truth table above.

3.5. 2-to-1 Multiplexer (`nBitMux2to1`)

Instances in the design:

- **RegDst MUX** (3-bit): selects write register address : `instruction[18:16]` (I-type) vs. `instruction[13:11]` (R-type).
- **ALUSrc MUX** (8-bit): selects ALU operand B : `read_data2` vs. `sign_ext_out`.
- **MemtoReg MUX** (8-bit): selects register write data : `alu_result` vs. `mem_read_data`.
- **Branch MUX** (8-bit): selects next PC : `pc_plus4` vs. `branch_target`.
- **Jump MUX** (8-bit): selects final next PC : `pc_branch_out` vs. `jump_addr`.
- **Output MUX tree** (8-bit, 7 instances): implements the 8-to-1 debug multiplexer for `MuxOut`.

3.6. Register File (`register_file`)

The register file contains eight 8-bit registers addressed by 3-bit fields. It provides:

- Two simultaneous read ports (`read_data1`, `read_data2`)
- One write port (`write_data`).

Port connections in the top-level:

- `read_reg1` ← `instruction[23:21]` (rs)
- `read_reg2` ← `instruction[18:16]` (rt)
- `write_reg` ← `reg_write_addr` (output of RegDst MUX)

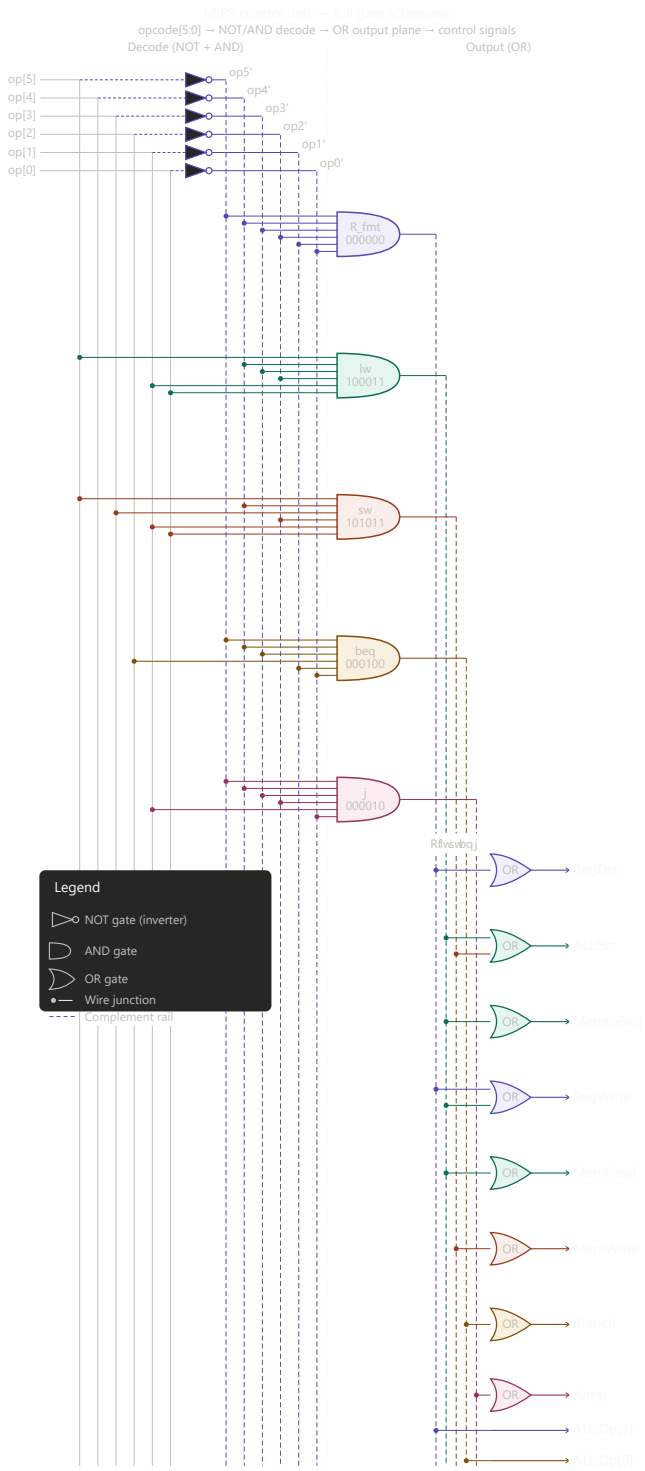


Figure 1: Control unit — full gate schematic (NOT/AND decode plane and OR output plane).

3.7. Sign Extension Unit (`sign_extended`)

The sign extension unit takes the 16-bit immediate field `instruction[15:0]` and produces an 8-bit output. Since the datapath is only 8 bits wide, the unit effectively truncates the upper bits while preserving the sign. The lower 8 bits of the immediate field pass through, and the MSB of

the 8-bit output replicates `instruction[7]` to maintain signed representation within the reduced data width.

3.8. ALU Control (`ALU_control`)

The ALU control unit generates the 3-bit `Operation` signal from the 2-bit `ALUOp` (from the main control unit) and the 6-bit `funct` field `instruction[5:0]`:

ALUOp	Funct	Operation	Action
`00`	×	`010`	Add (lw/sw address)
`01`	×	`110`	Subtract (beq comparison)
`10`	`100000` (32)	`010`	Add
`10`	`100010` (34)	`110`	Subtract
`10`	`100100` (36)	`000`	AND
`10`	`100101` (37)	`001`	OR
`10`	`101010` (42)	`111`	Set less than

Table 3: ALU control signal generation.

3.9. N-Bit ALU (`nbit_ALU`)

The 8-bit ALU accepts two 8-bit operands and a 3-bit control code. It supports five operations: AND, OR, addition, subtraction, and set-less-than (SLT). The `o_Zero` flag is asserted when the ALU result is all zeros.

The SLT operation reuses the subtractor and routes the sign bit (MSB of the difference) to bit 0 of the result, with all other bits set to zero.

3.10. Data Memory (`data_memory`)

The data memory is a 256×8 synchronous RAM instantiated using the Altera LPM_RAM_DQ megafunction. It provides a single read/write port:

- **Write:** When `wren = '1'`, data on the `data` port is written to the specified `address` on the clock edge.
- **Read:** The content at `address` is always available on `q`.

4. Datapath Interconnection

This section traces the signal flow through the four stages of the single-cycle datapath.

4.1. Fetch Stage

1. The PC register (`U_PC`) outputs `pc_current`.
2. The PC+4 adder (`U_PC_ADD`) computes `pc_plus4 = pc_current + 4`.
3. The instruction memory (`U_IMEM`) is addressed by `pc_next` and outputs the 32-bit `instruction`.
4. `pc_next` is determined by the Jump MUX at the end of the datapath (feedback path).

4.2. Decode Stage

1. The opcode `instruction[31:26]` feeds the control unit, generating all control signals.

2. The RegDst MUX selects the write register: `instruction[18:16]` (rt) for I-type or `instruction[13:11]` (rd) for R-type.
3. The register file reads `rs = instruction[23:21]` and `rt = instruction[18:16]`, outputting `read_data1` and `read_data2`.
4. The sign extension unit converts `instruction[15:0]` to the 8-bit `sign_ext_out`.

4.3. Execute Stage

1. The ALU control unit combines `ALUOp` and `instruction[5:0]` to produce the 3-bit ALU operation code.
2. The ALUSrc MUX selects ALU input B: `read_data2` (R-type) or `sign_ext_out` (I-type).
3. The ALU computes `alu_result` and asserts `alu_zero` if the result is zero.

4.4. Memory and Write-Back Stage

1. The data memory is addressed by `alu_result`. For `sw`, `read_data2` is written; for `lw`, the memory output `mem_read_data` is read.
2. The MemtoReg MUX selects `reg_write_data`: `alu_result` (R-type) or `mem_read_data` (`lw`).
3. The branch offset is computed by shifting `sign_ext_out` left by 2: `branch_offset = sign_ext_out & "00"`.
4. The branch adder computes `branch_target = pc_plus4 + branch_offset`.
5. `take_branch = Branch AND alu_zero` selects between `pc_plus4` and `branch_target`.
6. The jump address is formed as `jump_addr = instruction[5:0] & "00"` (shifted left by 2).
7. The Jump MUX produces the final `pc_next`, completing the feedback loop.

4.5. Output Debug Multiplexer

A tree of seven 2-to-1 multiplexers implements an 8-to-1 selection controlled by `ValueSelect[2:0]` to route various internal signals to the `MuxOut` output for debugging purposes.

Instr.	Op	RegDst	ALUSrc	Mem-toReg	Reg-Write	Mem-Read	MemWrite	Branch	Jump	ALUOp
add	000000	1	0	0	1	0	0	0	0	10
sub	000000	1	0	0	1	0	0	0	0	10
and	000000	1	0	0	1	0	0	0	0	10
or	000000	1	0	0	1	0	0	0	0	10
slt	000000	1	0	0	1	0	0	0	0	10
lw	100011	0	1	1	1	1	0	0	0	00
sw	101011	–	1	–	0	0	1	0	0	00
beq	000100	–	0	–	0	0	0	1	0	01
jump	000010	–	–	–	0	0	0	0	1	--

Table 4: Complete control signal settings per instruction type.

Cycle	PC	Instr.	ALU Result	Zero	Branch	MemW	RegW
1	00	lw $2,0	00	0	0	0	1
2	04	lw $3,1	01	0	0	0	1
3	08	sub $1,$2,$3	AB	0	0	0	1
4	0C	or $4,$1,$3	AB	0	0	0	1
5	10	sw $4,3	03	0	0	1	0
6	14	add $1,$2,$3	FF	0	0	0	1
7	18	sw $1,4	04	0	0	1	0
8	1C	lw $2,3	03	0	0	0	1
9	20	lw $3,4	04	0	0	0	1
10	24	j 11	–	–	0	0	0
11	2C	beq $1,$2,-8	54	0	1	0	0

Table 5: Cycle-by-cycle execution trace of the benchmark program.

4.6. Cycle-by-Cycle Trace

5. Timing Analysis and CPU Execution Time

5.1. Critical Path Analysis

The longest path in the single-cycle processor determines the minimum clock period. The critical path is the **load word** instruction, which traverses:

$$T_{\text{cycle}} = t_{\text{PC}} + t_{\text{IMEM}} + t_{\text{RF,read}} + t_{\text{MUX}} + t_{\text{ALU}} + t_{\text{DMEM}} + t_{\text{MUX}} + t_{\text{RF,setup}}$$

5.1.1. Gate-Level Delay of the 8-Bit Ripple-Carry Adder

Each 1-bit full adder has a carry propagation delay of 2 gate delays (one XOR + one AND-OR). For an 8-bit ripple-carry adder:

$$t_{\text{adder}} = 2 \times 8 \times 0.01 \text{ ns} = 0.16 \text{ ns}$$

5.1.2. ALU Worst-Case Delay

The ALU includes the adder/subtractor plus output multiplexing. With the B-input XOR for subtraction (1 gate) and the 4-to-1 output mux (2 gate levels):

$$t_{\text{ALU}} = t_{\text{XOR}} + t_{\text{adder}} + t_{\text{mux}} = 0.01 + 0.16 + 0.02 = 0.19 \text{ ns}$$

5.1.3. Total Critical Path

Using the given assumptions (memory: 2 ns, register file: 1 ns, gate: 0.01 ns):

Component	Delay (ns)	Notes
PC register (clk→q)	≈ 0.02	2 gate delays
Instruction memory	2.00	Given
Register file (read)	1.00	Given
ALUSrc MUX	0.02	2 gate delays
ALU	0.19	Calculated above
Data memory	2.00	Given
MemtoReg MUX	0.02	2 gate delays
Total	**5.25**	

Table 6: Critical path delay breakdown.

$$T_{\text{cycle}} \approx 5.25 \text{ ns} \quad \rightarrow \quad f_{\max} = \frac{1}{T_{\text{cycle}}} \approx 189.8 \text{ MHz}$$

5.2. CPU Execution Time Calculation

Following the methodology from our textbook, the CPU execution time is:

$$T_{\text{CPU}} = \text{IC} \times \text{CPI} \times T_{\text{cycle}}$$

where:

- **IC** (Instruction Count) = 11 instructions in the benchmark (before reaching NOP/halt)
- **CPI** = 1 (single-cycle design, every instruction takes exactly one clock cycle)

$$T_{\text{CPU}} = 11 \times 1 \times 5.27 \text{ ns} = 57.97 \text{ ns}$$

Since CPI = 1 for all instructions in a single-cycle design, the total CPU execution time remains:

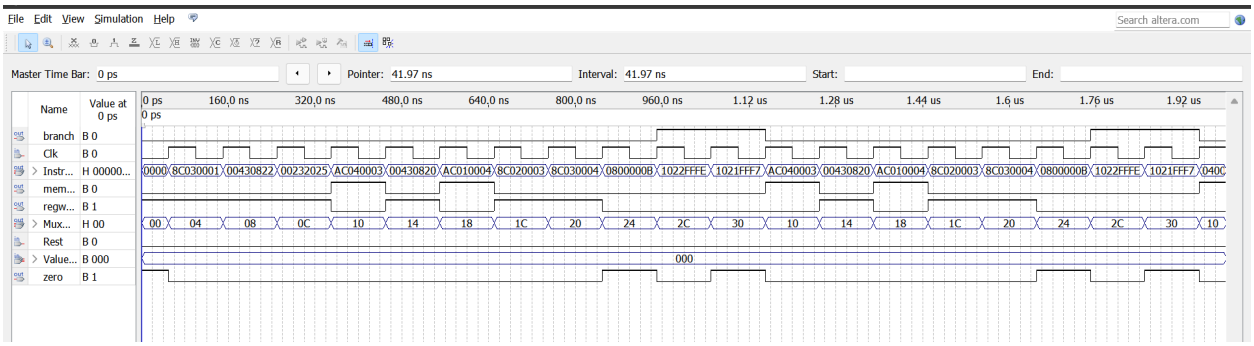
$$T_{\text{CPU}} = 11 \times 5.27 \text{ ns} \approx \mathbf{58 \ \ ns}$$

6. Simulation Results

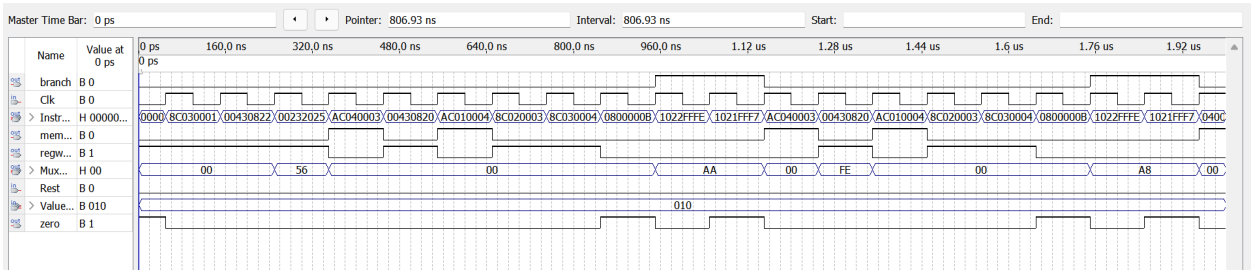
Functional and timing simulations were performed in Quartus II 13.1. The simulation verified correct execution of the benchmark program by observing the following signals:

- `InstructionOut[31:0]` : confirmed correct instruction fetch sequence.
- `MuxOut[7:0]` with varying `ValueSelect` :confirmed correct PC progression, ALU results, register file read/write data, and control signal values.
- `BranchOut`, `ZeroOut`, `MemWriteOut`, `RegWriteOut` : confirmed correct control signal assertion per instruction.

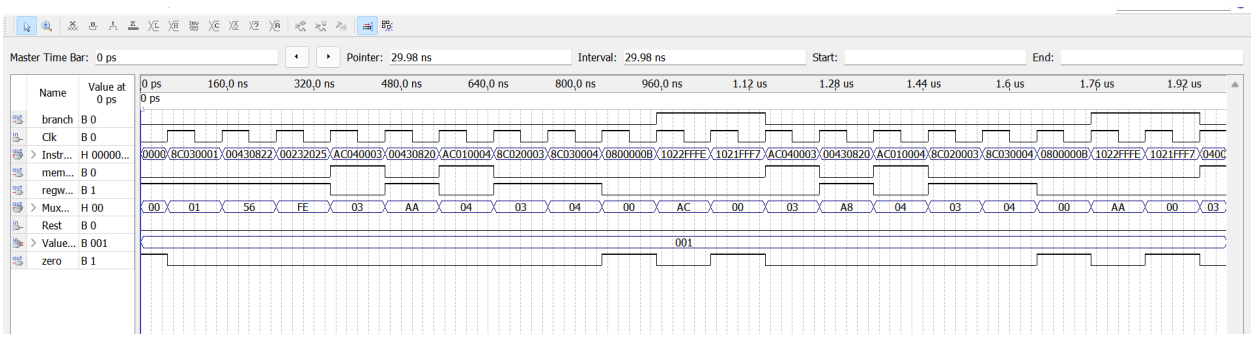
PC (ValueSelect = 000)



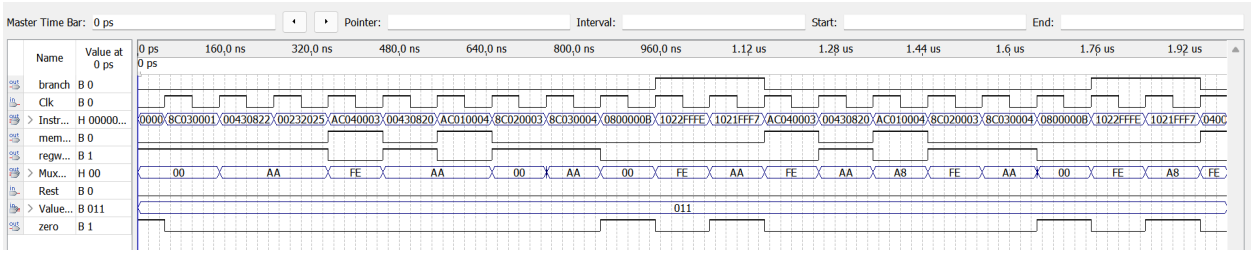
ALU Result (ValueSelect = 010)



Read Data 1 (ValueSelect = 001)



Read Data 2 (ValueSelect = 011)



Write Data (ValueSelect = 100)

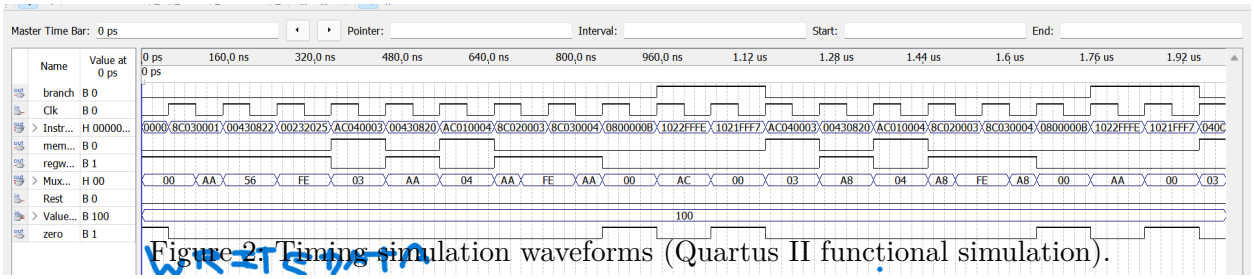


Figure 2: Timing simulation waveforms (Quartus II functional simulation).

7. Design Challenges and Solutions

Several design obstacles were encountered during implementation:

1. **Instruction Memory Synchronization:** The Altera LPM_ROM contains an internal address register that introduces a one-cycle latency. If `pc_current` were used as the ROM address, the fetched instruction would lag the PC by one cycle. This was resolved by connecting `pc_next` to the ROM address input, ensuring the ROM's internal register and the PC register both latch the same address on the same rising edge.
2. **Data Memory Write Timing:** For store instructions, the ALU result (memory address) and `read_data2` (write data) are only valid after the rising clock edge propagates through the datapath. Writing on the same rising edge would capture stale values. The solution was to clock the data memory on the **falling** edge of `GClock` (`NOT GClock`).

8. Conclusion

A fully functional 8-bit single-cycle RISC processor was successfully designed, implemented in structural VHDL, and verified in simulation. The processor correctly executes all supported MIPS instructions : `lw`, `sw`, `add`, `sub`, `and`, `or`, `slt`, `beq`, and `jump`.

The critical path analysis gives a maximum clock frequency of approximately 190 MHz, with the load word instruction defining the longest delay path through the PC, instruction memory, register file, ALU, data memory, and write-back multiplexer. The CPU execution time for the 11-instruction benchmark program is approximately 58 ns.

Note: The complete VHDL source code for all components is included in the github repository: https://github.com/MannP5/CEG3156_LAB_GROUP_4